

MACHINE LEARNING FUNDAMENTALS SERIES

Convolutional Neural Networks (CNNs): Principles & Backpropagation

Architecture Components, Worked Numerical Example, Filter Gradient Convolution, and Pooling Backprop

1. Introduction & Limitations of Traditional Neural Networks

Traditional Fully Connected (FC) Neural Networks process input images by flattening two-dimensional pixel grids into a single 1D vector of input nodes. While this approach works for small synthetic images (e.g., 6×6 pixels = 36 inputs), it fails on real-world images due to three critical drawbacks:

1. **Parameter Explosion:** A modest 100×100 RGB image contains 30,000 values. Connecting every input to a hidden layer with 1,000 nodes requires estimating **30,000,000** weights for that layer alone, leading to severe overfitting.
2. **Sensitivity to Spatial Shifts:** FC networks learn static connection weights for fixed pixel coordinates. If a pattern (e.g., the letter 'X' or 'O') is shifted right by just 1 pixel, different input nodes are activated, causing classification failure.
3. **Disregard of Local Spatial Correlations:** Nearby pixels in natural images are highly correlated (forming edges, textures, and shapes). Flattening destroys 2D spatial relationships.

Convolutional Neural Networks (CNNs) overcome these barriers by introducing three core principles: *Local Receptive Fields* (processing local pixel patches), *Weight Sharing* (sliding the same filter across the whole image), and *Spatial Downsampling* (pooling).

2. Comprehensive CNN Terminology & Architecture Components

A standard CNN architecture consists of several specialized layers, detailed below:

- **Filter / Kernel (W_f):** A small square matrix of weights (commonly 3×3 or 5×5). The filter slides across the input image to extract local visual patterns (e.g., vertical lines, corners). All spatial positions share the exact same filter weights.
- **Convolution Operation ($X * W_f$):** The mathematical operation of sliding the filter over the image, computing the element-wise product (dot product) of overlapping pixels, and summing them together.
- **Stride (S):** The step size (number of pixels) the filter shifts in each step across the image grid (typically $S = 1$ or $S = 2$).
- **Padding (P):** Adding extra pixel rows/columns (usually zeros) around the image border. *Valid Padding* ($P = 0$) reduces spatial size; *Same Padding* preserves input dimensions.
- **Feature Map (Z):** The output grid produced by convolving a filter over the input and adding a bias scalar b_f . Given input W , filter size K , padding P , and stride S , output dimension O is:

$$O = \left\lfloor \frac{W - K + 2P}{S} \right\rfloor + 1 \quad (1)$$

- **ReLU Activation:** Element-wise non-linear function $\text{ReLU}(z) = \max(0, z)$ applied to the feature map, setting negative values to zero while leaving positive values unchanged.
- **Pooling Layer (Max Pooling vs. Average Pooling):** Downsampling layer that reduces spatial dimensions to make features invariant to small shifts and lower computational cost.
 - **Max Pooling:** Selects the maximum value in each $K_p \times K_p$ window (capturing dominant feature responses).
 - **Average Pooling:** Computes the mean value of all pixels in each $K_p \times K_p$ window.
- **Fully Connected (FC) / Dense Layer:** Flattens the final downsampled feature maps into a 1D vector and routes them through a traditional feedforward network for final classification.
- **SoftMax / ArgMax Layer:** Output activation converting raw classification logits into normalized class probabilities (SoftMax) or discrete predictions (ArgMax).

3. Step-by-Step Worked Example (6×6 Image to Classification)

Consider the minimalist CNN model from the StatQuest example: a 6×6 binary image (X), a single 3×3 filter (W_f), stride $S = 1$, valid padding $P = 0$, followed by 2×2 Max Pooling ($S = 2$), flattening, and a Fully Connected output layer.

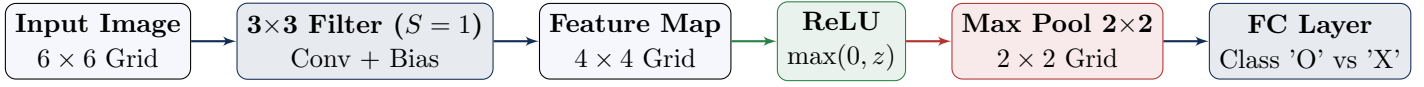


Figure 1: Complete Transformation Pipeline of the Benchmark Minimalist CNN Architecture.

3.1 Numerical Step-by-Step Walkthrough

1. Spatial Dimension Calculation: For input $W = 6$, filter $K = 3$, padding $P = 0$, stride $S = 1$:

$$O_{\text{conv}} = \frac{6 - 3 + 0}{1} + 1 = 4 \implies \text{Feature Map Size} = 4 \times 4 \quad (2)$$

2. Convolution Calculation (Sample Overlap Patch at top-left corner): Let top-left 3×3 patch of image X be $X_{1:3,1:3} = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ and learned filter $W_f = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 1 \end{bmatrix}$ with bias $b_f = -1.0$:

$$\text{Dot Product} = (0 \cdot 1) + (1 \cdot 0) + (0 \cdot 1) + (1 \cdot 0) + (1 \cdot 1) + (1 \cdot 0) + (0 \cdot 1) + (1 \cdot 0) + (0 \cdot 1) = 1 \quad (3)$$

$$z_{1,1} = \text{Dot Product} + b_f = 1 + (-1.0) = 0.0 \quad (4)$$

Sliding the filter across all 16 positions generates the 4×4 raw feature map Z .

3. ReLU Non-linear Activation: Applying $\text{ReLU}(z) = \max(0, z)$ to every element in Z produces the 4×4 activated feature map A :

$$A = \begin{bmatrix} 2.5 & 0.0 & 0.0 & 1.2 \\ 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 \\ 1.8 & 0.0 & 0.0 & 2.1 \end{bmatrix} \quad (5)$$

4. Max Pooling (2×2 Window, Stride $S = 2$): The 4×4 grid is divided into four non-overlapping 2×2 quadrants:

- Top-Left Quadrant $\begin{bmatrix} 2.5 & 0.0 \\ 0.0 & 0.0 \end{bmatrix} \implies \max = 2.5$, Top-Right Quadrant $\begin{bmatrix} 0.0 & 1.2 \\ 0.0 & 0.0 \end{bmatrix} \implies \max = 1.2$
- Bottom-Left Quadrant $\begin{bmatrix} 0.0 & 0.0 \\ 1.8 & 0.0 \end{bmatrix} \implies \max = 1.8$, Bottom-Right Quadrant $\begin{bmatrix} 0.0 & 0.0 \\ 0.0 & 2.1 \end{bmatrix} \implies \max = 2.1$

This downsamples the activated map to a 2×2 pooled matrix $P = \begin{bmatrix} 2.5 & 1.2 \\ 1.8 & 2.1 \end{bmatrix}$.

5. Flattening & Fully Connected Layer: Flattening P yields 4 input nodes $[2.5, 1.2, 1.8, 2.1]^T$. Multiplying by FC weights and applying Softmax yields class probabilities:

$$\hat{y}_{\text{'O'}} = 0.92, \quad \hat{y}_{\text{'X'}} = 0.08 \implies \text{Classified as Letter 'O'} \quad (6)$$

4. CNN Backpropagation: Filter Weight Gradient Updates

During Backpropagation, the network receives an error matrix $\delta^{(l)}$ of size 4×4 from the subsequent layer (after backpropagating through FC and Pooling layers). We must calculate the gradient of loss $\mathcal{L}$ with respect to each weight in the 3×3 filter W_f .

Mathematical Core: Filter Gradient via Cross-Correlation

The gradient of Loss $\mathcal{L}$ with respect to filter W_f is computed by ****convolving** (or cross-correlating) the 6×6 original input image X with the 4×4 error matrix $\delta^{(l)}$:

$$\frac{\partial \mathcal{L}}{\partial W_f} = X * \delta^{(l)} \quad (7)$$

4.1 Dimension Verification & Mathematical Proof

- **Input Image X Shape:** 6×6
- **Error Matrix $\delta^{(l)}$ Shape:** 4×4
- **Cross-Correlation Output Shape:** $(6 - 4 + 1) \times (6 - 4 + 1) = 3 \times 3$

Notice how convolving a 6×6 image with a 4×4 error matrix yields an output of size **exactly 3×3** , matching the shape of filter W_f !

4.2 Detailed Derivation for Filter Weight $w_{r,c}$

For a specific filter weight $w_{r,c}$ (where $r, c \in \{1, 2, 3\}$), $w_{r,c}$ contributed to every single element (i, j) in feature map Z according to:

$$z_{i,j} = \sum_{r=1}^3 \sum_{c=1}^3 x_{i+r-1, j+c-1} \cdot w_{r,c} + b_f \quad (8)$$

By the Multivariable Chain Rule, the total gradient for weight $w_{r,c}$ is the sum of partial derivatives across all $4 \times 4 = 16$ spatial locations where $w_{r,c}$ was applied:

$$\frac{\partial \mathcal{L}}{\partial w_{r,c}} = \sum_{i=1}^4 \sum_{j=1}^4 \frac{\partial \mathcal{L}}{\partial z_{i,j}} \cdot \frac{\partial z_{i,j}}{\partial w_{r,c}} = \sum_{i=1}^4 \sum_{j=1}^4 \delta_{i,j}^{(l)} \cdot x_{i+r-1, j+c-1} \quad (9)$$

This summation formula is precisely the cross-correlation of X with $\delta^{(l)}$.

4.3 Bias Gradient & Weight Update Step

The filter bias b_f affects every cell $z_{i,j}$ with a derivative of 1 ($\frac{\partial z_{i,j}}{\partial b_f} = 1$). Its gradient is simply the sum of all elements in the error matrix:

$$\frac{\partial \mathcal{L}}{\partial b_f} = \sum_{i=1}^4 \sum_{j=1}^4 \delta_{i,j}^{(l)} \quad (10)$$

Using learning rate η , filter weights and bias are updated via Gradient Descent:

$$W_f^{(\text{new})} = W_f^{(\text{old})} - \eta \cdot \frac{\partial \mathcal{L}}{\partial W_f}, \quad b_f^{(\text{new})} = b_f^{(\text{old})} - \eta \cdot \frac{\partial \mathcal{L}}{\partial b_f} \quad (11)$$

5. Backpropagation through Pooling Layers

Pooling layers have no learnable parameters, but they must **route incoming gradients backward** to reconstruct the 4×4 error matrix δ^{conv} for the preceding convolutional layer.

5.1 Max Pooling Backpropagation (ArgMax Routing)

- **Forward Mechanism:** Selects max value in each 2×2 window, storing the *ArgMax coordinates* (r^*, c^*) .
- **Backward Mechanism:** Gradient is routed **exclusively** to the position of the maximum value that produced the output. All non-max positions receive a gradient of 0.

$$\delta_{i,j}^{\text{conv}} = \begin{cases} \delta_{u,v}^{\text{pool}} & \text{if } (i, j) = \text{ArgMax within window } (u, v) \\ 0 & \text{otherwise} \end{cases} \quad (12)$$

Example: Given incoming 2×2 gradient $\delta^{\text{pool}} = \begin{bmatrix} 0.8 & -0.4 \\ 0.5 & 1.1 \end{bmatrix}$ and stored ArgMax locations at top-left of each quadrant, the reconstructed 4×4 error matrix δ^{conv} is:

$$\delta^{\text{conv}} = \begin{bmatrix} \mathbf{0.8} & 0.0 & 0.0 & \mathbf{-0.4} \\ 0.0 & 0.0 & 0.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 \\ \mathbf{0.5} & 0.0 & 0.0 & \mathbf{1.1} \end{bmatrix} \quad (13)$$

5.2 Average Pooling Backpropagation (Uniform Gradient Splitting)

- **Forward Mechanism:** Computes Average = $\frac{1}{K_p \times K_p} \sum x_{i,j}$ across $K_p \times K_p$ window (e.g., $2 \times 2 \implies \frac{1}{4}$).
- **Backward Mechanism:** Every pixel in the window contributed equally to the average. Thus, the incoming gradient $\delta_{u,v}^{\text{pool}}$ is **distributed equally (split by 1/4)** across all 4 window locations:

$$\delta_{i,j}^{\text{conv}} = \frac{1}{K_p \times K_p} \cdot \delta_{u,v}^{\text{pool}} = \frac{1}{4} \cdot \delta_{u,v}^{\text{pool}} \quad (14)$$

Example: For top-left quadrant gradient $\delta_{1,1}^{\text{pool}} = 0.8$, each of the 4 pixels in that 2×2 region receives $\frac{0.8}{4} = 0.2$.

6. Summary Comparison of CNN Layer Operations

Table 1: Forward Pass Computation and Backward Gradient Flow in CNN Layers.

CNN Layer	Forward Pass Operation	Backward Pass Gradient Flow
Convolution	Feature extraction via sliding dot product: $Z = X * W_f + b_f$	Filter gradient: $\frac{\partial \mathcal{L}}{\partial W_f} = X * \delta^{(l)}$; Input gradient: $\delta^{(l-1)} = \delta^{(l)} * \text{Rot180}(W_f)$
ReLU Activation	Non-linear clipping: $A = \max(0, Z)$	Gradient pass-through if $z > 0$; zeroed out if $z \leq 0$: $\delta^{(l-1)} = \delta^{(l)} \odot \mathbb{I}(Z > 0)$
Max Pooling	Select maximum value in $K_p \times K_p$ window	Route gradient $\delta^{(l)}$ exclusively to ArgMax location; 0 elsewhere
Average Pooling	Compute mean value in $K_p \times K_p$ window	Distribute gradient equally: $\delta^{(l-1)} = \frac{1}{K_p^2} \cdot \delta^{(l)}$ across all window pixels
Fully Connected	Dense matrix multiplication: $\hat{Y} = W_{fc} A_{\text{flat}} + b_{fc}$	Standard backpropagation: $\frac{\partial \mathcal{L}}{\partial W_{fc}} = \delta^{(l)} A_{\text{flat}}^T$